

АРХИТЕКТУРА СИСТЕМЫ

«Рейс-Контроль»

Предложение по архитектуре мобильного приложения, серверной части и хранения данных

Версия документа 1.1
02 октября 2026 г.

Статус: предложение к согласованию

Основание: техническое задание «Рейс-Контроль» v4.0 от 02.10.2026

Конфигурация как данные • событийный журнал • offline-first
порты и адаптеры • мультиплатформенность

2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
Карта документа	4
1. Резюме	4
2. Подход к проектированию	5
2.1. Основание	5
2.2. Что взято из каждого подхода	5
2.3. Принцип синтеза	5
3. Архитектурные принципы	6
4. Архитектура системы	6
4.1. Общая схема	6
4.2. Почему API и воркеры разделены	7
4.3. Загрузка фото — двухфазная, минуя API	7
5. Хранение данных.....	7
5.1. Выбор СУБД: PostgreSQL managed	7
5.2. Разделение данных на три категории	8
5.3. Схема events (ядро).....	8
5.4. Основные группы таблиц	9
5.5. Object Storage: раскладка	9
5.6. Локальное хранилище на устройстве	10
5.7. Целостность и восстановление.....	10
5.8. Периодический трекинг (ГЛОНАСС-аналог)	10
6. Конфигурация как данные	11
6.1. Примитивы (стартовый набор)	11
6.2. Пример декларативного чек-листа	11
6.3. Как это закрывает требования ТЗ.....	12
6.4. Жизненный цикл конфигурации	12
7. Синхронизация и офлайн-режим.....	12
7.1. Жизненный цикл события	12
7.2. Идемпотентность	13
7.3. Приоритеты загрузки	13
7.4. Очистка на клиенте.....	13
7.5. Разделение состояний	13
7.6. Фоновые задачи и очередь.....	13
8. Время, координаты, доказательность	13
8.1. Время как набор атрибутов	13
8.2. Геолокация через порт	14
8.3. Периодический трекинг (ГЛОНАСС-аналог)	14
8.4. Водяной знак	15

9. Уведомления и интеграции	15
9.1. Каналы уведомлений	15
12 ракурсов не влезают в один Telegram-альбом — несколько сообщений или ссылка в кабинет.	15
9.2. Telegram как канал доставки, не хранилище	15
12 ракурсов — несколько сообщений или ссылка в кабинет.	15
9.3. Transactional outbox	15
9.4. Юридический момент	16
9.5. RuStore	16
9.6. Точки расширения	16
10. Структура репозитория	16
10.1. Общая структура (моногеро)	16
10.2. Android — модули	17
10.3. iOS — модули (закладывается)	17
10.4. Backend — правило зависимостей	17
11. Версии и совместимость	17
11.1. Независимые версии	17
11.2. Правила	18
11.3. Каналы обновления	18
11.4. Миграции	18
12. Этапы работ	18
12 ракурсов, обязательные документы.	19
13. Критерии приёмки	20
13.1. Из ТЗ (базовые)	20
12 ракурсов на каждую машину, событие не сохраняется до окончания.	20
13.2. Дополнительные (критичные)	20
13.3. Из требований гибкости	20
14. Открытые вопросы к заказчику	20
14.1. Расчёт объёмов (важно)	21
12 ракурсов × 8 машин + ~10 документов ≈ 110–130 фото на рейс.	21
30 рейсов/мес → ~2 ГБ/мес.	21
200 рейсов/мес → ~13 ГБ/мес, ~160 ГБ/год + исходящий трафик на просмотр логистом.	21
15. Антипаттерны	21
16. Итоговая трассировка на требования ТЗ	22

Оглавление обновляется автоматически при открытии документа в Word.

Карта документа

Резюме
 Подход к проектированию
 Архитектурные принципы
 Архитектура системы
 Хранение данных
 Конфигурация как данные
 Синхронизация и офлайн-режим
 Время, координаты, доказательность
 Уведомления и интеграции
 Структура репозитория
 Версии и совместимость
 Этапы работ
 Критерии приёмки
 Открытые вопросы к заказчику
 Антипаттерны
 Итоговая трассировка на требования ТЗ

1. Резюме

ТЗ «Рейс-Контроль» по сути требует не «приложение с фото», а платформу фиксации событий с конфигурируемым чек-листом и офлайн-синхронизацией. Главный драйвер архитектуры — разделы 6 и 7 ТЗ (гибкость, готовность к обновлениям, гибкость хранения).

- Настоящий документ — синтез двух подходов к архитектуре:

Подход «гибкость» — приоритет отдаётся платформенным решениям, которые выдерживают грандиозные обновления через годы: событийный журнал, проекции, config-as-data, порты и адаптеры.

Подход «прагматизм» — приоритет отдаётся реализуемости в срок 2–2,5 месяца и простоте для команды: PostgreSQL вместо YDB, ограниченный набор примитивов вместо универсального конструктора, честные критерии приёмки.

- Философия гибрида: строим правильно то, что невозможно переделать потом; не строим то, что можно добавить позже без переделки.
- Ключевые решения:
- Конфигурация и события — как данные, а не как код.
- Иммутабельный журнал событий + проекции для чтения.
- Офлайн-first синхронизация с идемпотентностью.
- Порты и адаптеры на всех внешних границах (геолокация, уведомления, хранилище, обновления, интеграции).
- PostgreSQL managed вместо YDB.
- Ограниченный набор примитивов вместо универсального конструктора.
- Мультиплатформенность: Android — основной клиент на старте, iOS — закладывается сразу.
- Каналы обновления: APK напрямую, RuStore, TestFlight, App Store / Enterprise.
- Периодический трекинг (ГЛОНАСС-аналог) — бесплатными методами (WorkManager / BGTaskScheduler).

2. Подход к проектированию

2.1. Основание

- Техническое задание «Рейс-Контроль» v4.0 от 02.10.2026.

2.2. Что взято из каждого подхода

Решение	Источник	Почему
Событийный журнал + проекции	Подход «гибкость»	Единственный способ менять модель без потери накопленного
Config-as-data + snapshot на рейс	Подход «гибкость»	Прямое закрытие п. 6.5 ТЗ
Строгая модель времени	Подход «гибкость»	Доказательность и офлайн
Порты и адаптеры на границах	Подход «гибкость»	GMS/HMS/Platform/ios, каналы, хранилища
Двухфазная загрузка файлов	Подход «гибкость»	Снимает гигабайты с API, докачка
cargo_units, trip_participants сразу	Подход «гибкость»	Иначе этапы 2 и подмены ломают доступы
Append-only + JSONB + expand/contract	Подход «гибкость»	Конкретные правила миграции
ADR, contract-first, IaC, три среды	Подход «гибкость»	Передача и сопровождение (п. 6.4, п. 17)
PostgreSQL managed, не YDB	Оба	Оба автора согласны
Простой набор примитивов	Подход «прагматизм»	Прагматизм пилота
Разделение состояний: событие / доставка / рейс	Подход «прагматизм»	То, что чаще всего путают
Дополнительные критерии приёмки	Подход «прагматизм»	Обновление с неотправленными фото, повтор → одно событие
Реалистичные сроки	Подход «прагматизм»	2–2,5 мес — это пилот
152-ФЗ, копия фото в Telegram	Подход «прагматизм»	Юридический риск
Transactional outbox	Оба	Оба предлагают
Мультиплатформенность (Android + iOS)	Новое	Решение заказчика от 02.10.2026
RuStore как канал	Новое	Решение заказчика от 02.10.2026
Периодический трекинг (ГЛОНАСС-аналог)	Новое	Решение заказчика от 02.10.2026

2.3. Принцип синтеза

Формулировка: «Строй правильно то, что невозможно переделать потом. Не строй то, что можно добавить позже без переделки.»

- Расшифровка:

Невозможно переделать потом: структура хранения фактов, модель времени, конфигурация как данные, идемпотентность, мультиплатформенность. Это закладывается на этапе 1 и стоит копейки сейчас.

Можно добавить позже: новые типы событий, новые ракурсы, новые интеграции, новые каналы уведомлений, iOS-клиент. Это добавляется через справочники и адаптеры без переписывания.

3. Архитектурные принципы

- Принципы, которые делают систему гибкой по п. 6–7 ТЗ.
- Принцип 1. Config-as-data (server-driven).
- Типы событий, чек-листы, ракурсы, документы, правила завершения — строки в БД, а не код. Приложение — интерпретатор ограниченного набора примитивов.
- Принцип 2. Снимок конфигурации на рейс.
- При старте рейса фиксируется иммутабельный `config_snapshot_id`. Правка чек-листа в середине рейса не меняет уже начатые рейсы.
- Принцип 3. Проекция конфига под версию клиента.

Сервер знает `min_app_version` каждого элемента и отдаёт старой версии только поддерживаемое. Неизвестный примитив на клиенте — мягкая деградация, никогда краш.

- Принцип 4. Событие — неизменяемый факт (append-only).
- Источник истины — журнал событий; таблицы-проекции строятся из него. Смена структуры = новая проекция и пересборка, старые данные не переписываются.
- Принцип 5. Аддитивность контрактов.
- API: только добавление полей, никогда не удаление и не переименование. Каждое сообщение несёт `schema_version`.
- Принцип 6. Идемпотентность везде.

`client_event_id` (UUIDv7) на клиенте, `Idempotency-Key` на API, `attachment_id` на файлах. Повторная отправка ничего не дублирует — критично для плохой связи.

- Принцип 7. Физический путь файла ≠ источник правды.

Связь «рейс ↔ событие ↔ фото ↔ документ» живёт в БД по ID; раскладка в Object Storage — деталь реализации, а красивая раскладка по папкам (п. 11 ТЗ) — виртуальная, генерируется при экспорте.

- Принцип 8. Порты и адаптеры на границах.

Хранилище, уведомления, геолокация, камера, обновления, внешние системы (1С, ЭТрН) — интерфейсы. Новый канал или интеграция = новый адаптер, без правок ядра.

- Принцип 9. Флаги функций, минимальная версия и kill switch.
- Поэтапное включение и мгновенное отключение без релиза.
- Принцип 10. Расширяемость через миграции `expand/contract`.
- Добавили поле → пишем в оба → бэкфил → переключили чтение → удалили старое. Никогда «big bang» миграция со стопом.
- Принцип 11. Офлайн-first как архитектура, а не функция.
- Клиент — источник истины для фактов, сервер — для справочников, конфигов и статусов.
- Принцип 12. Документируемость.
- ADR на каждое ключевое решение — прямое требование п. 6.4 ТЗ («команда, не участвовавшая в разработке»).
- Принцип 13. Мультиплатформенность.

API не привязан к платформе. Android и iOS используют один `openapi.yaml`. Заголовок `X-Platform` различает клиентов. Поля `platform` — везде, где применимо.

4. Архитектура системы

4.1. Общая схема

- Клиенты:

- Android (Kotlin / Compose): офлайн-first ядро, Room + Outbox, интерпретатор конфига, CameraX, LocationProvider (GMS / HMS / Platform).
- iOS (Swift / SwiftUI) — закладывается, реализуется позже: Core Data, AVFoundation, Core Location.
- Веб-кабинет (React + TypeScript): SPA, статика в Object Storage + CDN, конфигуратор чек-листов, карточка рейса, галерея, логи, флаги, релизы.
- Telegram-боты: @reiscontrol_bot, @reiscontrol_devbot.
- Серверная часть:

API (FastAPI, stateless): авторизация, синхронизация, заявки, рейсы, события, конфигурация, вложения, треки. API отвечает за 3 секунды и не занимается ресайзом.

- Очередь (YMQ или PostgreSQL): transactional outbox.
- Воркеры (контейнеры): превью, водяные знаки, PDF/акты, экспорт ZIP, проекции, уведомления.
- Scheduler (cron): реконсиляция, lifecycle, пересборка проекций, рассылки.
- Хранилища:
- PostgreSQL managed: справочники, события (append-only), проекции, очередь, треки.
- Yandex Object Storage: файлы (raw / derived / docs / exports / manifests / apk / ipa).
- Внешние каналы:
- Telegram Bot API: логист и dev-bot.
- FCM / HMS: push для Android.
- APNs: push для iOS (позже).
- Yandex Cloud: хостинг, секреты, мониторинг.

4.2. Почему API и воркеры разделены

API отвечает за 3 секунды и не занимается ресайзом; всё тяжёлое — асинхронно и ретраится независимо. Это даёт горизонтальное масштабирование: функции и контейнеры добавляются по нагрузке.

4.3. Загрузка фото — двухфазная, минуя API

- POST /v1/attachments:init → attachment_id + presigned PUT URL.
- Клиент льёт файл напрямую в Object Storage (с докачкой).
- POST /v1/attachments/{id}:commit с sha256, размером, метаданными водяного знака.
- Сервер проверяет наличие и хеш → stored, ставит задачу на превью и уведомление.
- Это снимает с API гигабайты трафика, убирает лимиты payload и позволяет докачку в плохой связи.

5. Хранение данных

5.1. Выбор СУБД: PostgreSQL managed

- Рекомендация — PostgreSQL managed.

Критерий	YDB serverless	PostgreSQL managed
Цена на пилоте	бесплатно	~1,5–4 тыс ₽/мес
Скорость разработки	медленнее (нет привычных ORM/миграций)	быстро (SQLAlchemy, Alembic, локально в Docker)
Аналитика и выгрузки	ограниченнее	полноценный SQL, оконные функции, JSONB + GIN
Экспертиза на рынке	дефицит	массовая (важно для п. 6.4)
Vendor lock-in	выше	ниже

Критерий	YDB serverless	PostgreSQL managed
Нагрузка проекта	избыточна	с запасом на годы при 30–200 рейсах/мес

- Разница в цене (~2 тыс ₽/мес) на порядок меньше стоимости недель, потерянных на непривычном стеке, и риска при передаче проекта.

5.2. Разделение данных на три категории

Категория	Примеры	Как хранится	Как меняется
Факты рейса	события (погрузка, выгрузка, фото), координаты, время	events — append-only	только новые записи, корректировка = новая запись со ссылкой
Справочники	users, orders, trips, trip_points, cargo_units, checklist_templates	обычные таблицы	CRUD, обычные миграции
Проекции	trip_timeline, trip_progress, trip_completeness, trip_tracks	материализованные представления	пересобираются из events

- Почему именно так:
- Факты нельзя «обновлять» — фото уже снято, координата уже получена. Append-only здесь не идеология, а физика.
- Справочники меняются нормально — водитель может переставить машину в автовозе, логист может переназначить рейс. Это CRUD.

Проекции — буфер гибкости. Когда через год понадобится новая структура «прогресс рейса», данные не мигрируются — пишется новая проекция и пересобирается из events. Старые записи не трогаются.

5.3. Схема events (ядро)

- Поля:

id (uuid, primary key)

client_event_id (uuid, UUIDv7, создан на телефоне)

device_id (uuid)

trip_id (uuid)

point_id (uuid, может быть null)

cargo_unit_id (uuid, может быть null)

event_type_code (text, например LOADING, UNLOADING)

payload_jsonb (jsonb, расширяемое содержимое)

payload_schema_version (int)

device_time_utc (timestampz)

device_tz_offset_min (int)

elapsed_realtime_ms (bigint, SystemClock.elapsedRealtime)

server_time_utc (timestampz)

clock_skew_ms (int)

time_trust (text: high / skewed / unknown)

lat, lon (numeric)

accuracy_m (int)

location_source (text: gms / hms / platform / ios)

state (text: accepted / rejected)

corrects_event_id (uuid, ссылка на корректируемое событие)

created_by (uuid)

app_version (text)

platform (text: android / ios)

created_at (timestampz)

- Ограничение: уникальный индекс (client_event_id, device_id).
- Правила:
- Никаких UPDATE после state='accepted'. Исправление = новая запись + corrects_event_id.
- payload_jsonb + payload_schema_version — новые поля пишутся, старые записи не переписываются.
- client_event_id + device_id — уникальный индекс для идемпотентности.
- Партиционирование по created_at (месяц) — для больших объёмов.

5.4. Основные группы таблиц

Группа	Таблицы
Доступ	users, roles, user_roles, driver_profiles, devices, refresh_tokens, access_scopes, user_consent
Заявки и рейсы	orders, trips, trip_participants, trip_points, point_types, cargo_types, trip_types, cargo_units
События (append-only)	events, event_types, event_payload_schemas
Конфигурация	checklist_templates, checklist_versions, checklist_steps, document_types, completion_policies, config_snapshots, rule_sets
Вложения	attachments, attachment_versions, attachment_kinds, derived_assets
Геолокация	location_tracks
Доставка и интеграции	outbox, inbox, notification_templates, notification_channels, notification_log, integration_jobs
Платформа	feature_flags, app_releases, audit_log, sync_cursors, export_jobs

5.5. Object Storage: раскладка

```
s3://reiscontrol-prod/
  raw/{trip_id}/{event_id}/{attachment_id}/original.jpg
  derived/{attachment_id}/w1920.jpg
  derived/{attachment_id}/thumb.jpg
  docs/{trip_id}/{attachment_id}/scan.pdf
  exports/{export_job_id}/trip-{trip_id}.zip
  manifests/{trip_id}/meta.json
  apk/reiscontrol-{versionName}-{versionCode}.apk
  ipa/reiscontrol-{versionName}-{versionCode}.ipa
```

Правила:

Ключ неизменяем (immutable). Меняете правила раскладки — новые файлы пишутся по-новому, старые остаются на месте, связь через attachment_id не рвётся (п. 7.2).

Раскладка из п. 11 ТЗ (/orders/123/loading_moscow/...) реализуется как виртуальная структура экспорта: сервер по ID собирает ZIP ровно с такими папками, правила раскладки – конфигурация (export_layout).

Lifecycle: raw/ → холодное хранилище через 90–180 дней, derived/ → удаление по необходимости.

Версионирование бакета включено; sha256 в БД позволяет в любой момент проверить целостность файла.

5.6. Локальное хранилище на устройстве

- Android:

Слой	Содержимое
Room	заявки и рейсы (кэш), config_snapshots, события и их состояния, outbox, справочники, логи
Файловая система	оригиналы фото и превью до подтверждённой загрузки; удаление только после state=accepted на сервере
Настройки	токены (EncryptedSharedPreferences), курсоры синхронизации, локальные флаги

- iOS (закладывается):

Слой	Содержимое
Core Data / Realm	то же, что Room
Файловая система	то же
Настройки	Keychain (вместо EncryptedSharedPreferences)

Правила: квота под фото (например, 2 ГБ) и предупреждение водителю; при нехватке места — сначала отправляем, потом чистим; миграции обязательны и тестируются.

5.7. Целостность и восстановление

- Бэкапы: PITR для PostgreSQL, версионирование бакета, экспорт «золотых» конфигураций в git.
- Задача планировщика — реконсилияция: «событие помечено uploaded, но файла нет или хеш не совпал» → alert в dev-бот.
- Проверка целостности рейса «одной кнопкой»: все обязательные шаги, все вложения на месте, хеши совпадают, временные метки непротиворечивы.
- Идемпотентный приём: повторная отправка того же client_event_id возвращает тот же результат.

5.8. Периодический трекинг (ГЛОНАСС-аналог)

- Таблица location_tracks:

Поле	Тип	Описание
id	uuid PK	—
device_id	uuid FK→devices.id	Устройство
user_id	uuid FK→users.id	Водитель
trip_id	uuid FK→trips.id	Рейс (nullable)
lat	numeric(9,6)	Широта
lon	numeric(9,6)	Долгота

Поле	Тип	Описание
accuracy_m	int	Точность
speed_mps	numeric	Скорость (если есть)
bearing_deg	numeric	Направление (если есть)
location_source	text	gms / hms / platform / ios
recorded_at	timestampz	Время записи
received_at	timestampz	Время приёма сервером

- Индексы: (device_id, recorded_at), (trip_id, recorded_at).
- Партиционирование: по recorded_at (месяц).
- Правила:
- Интервал: 15 минут (минимум Android для WorkManager).
- Только во время рейса. Вне рейса — выключен.
- Отдельное согласие пользователя (см. ADR-0010).
- Бесплатный метод: WorkManager (Android) / BGTaskScheduler (iOS) + foreground service с уведомлением.
- Отправка: батчами при появлении сети (POST /location/tracks).

6. Конфигурация как данные

6.1. Примитивы (стартовый набор)

- Не универсальный конструктор, а ограниченный набор примитивов.

Примитив	Что делает	Пример
photo_set	набор фото по ракурсам	12 ракурсов машины
document_set	набор документов	ТТН, CMR, путевой
number	числовой ввод	пробег, расход топлива
text	текстовый ввод	трек-номер, комментарий
confirm	подтверждение	«принял груз», «сдал груз»
signature	подпись	акт осмотра (этап 16)
geo_only	только координаты и время	прибытие на точку

- Новый тип события = строка в event_types + привязка к одному из примитивов. Новый ракурс = новая строка в checklist_steps. Кода — ноль.

6.2. Пример декларативного чек-листа

```
{
  "snapshot_id": "cs_01HX...",
  "trip_id": "tr_01HX...",
  "checklist_version": 14,
  "min_app_version": 1,
  "event_type": { "code": "LOADING", "primitive": "photo_set" },
  "scope": "per_cargo_unit",
  "steps": [
    { "code": "front_3_4", "type": "photo", "title": "Спереди слева", "required": true },
    { "code": "rear_3_4", "type": "photo", "title": "Сзади справа", "required": true },
    { "code": "vin_plate", "type": "photo", "title": "Табличка VIN", "required": true },
  ]
}
```

```

    { "code": "ttn",          "type": "document", "title": "ТТН", "required": true }
  ],
  "completion_rules": [
    { "rule": "all_required_steps_done" },
    { "rule": "min_photos", "count": 12, "per": "cargo_unit" },
    { "rule": "require_track_number" }
  ]
}

```

6.3. Как это закрывает требования ТЗ

Требование ТЗ	Механизм
Менять состав и порядок шагов без релиза (6.2)	правка checklist_steps + публикация новой версии шаблона
Новый тип события (6.2)	строка в event_types + привязка к примитиву
Новые ракурсы (16)	новые steps с type=photo, без кода
Новые типы вложений (7.2)	новый attachment_kind
Разные конфигурации для типов груза и клиентов (6.5)	checklist_template выбирается по cargo_type, trip_type, client_id
Старые версии приложения (6.3)	сервер фильтрует элементы по min_app_version; неизвестный примитив → мягкая деградация
Не ломать начатые рейсы (6.3)	рейс работает по своему config_snapshot
Отключить функцию без релиза (6.2)	feature_flags + kill switch

6.4. Жизненный цикл конфигурации

- Черновик → проверка → публикация → применение к новым рейсам → архивирование.
- Новые рейсы получают новую опубликованную версию чек-листа; активные продолжают по прежней. Менять их требования можно отдельной операцией с историей.

7. Синхронизация и офлайн-режим

7.1. Жизненный цикл события

Состояние	Где	Что означает	Что видит водитель
draft	телефон	черновик, шаги не выполнены	«Не закончено»
complete	телефон	все шаги выполнены	«Готово к отправке»
queued	телефон	в outbox, ждёт сеть	«В очереди»
uploaded	телефон → сервер	файлы загружены	«Отправлено»
accepted	сервер	принято	«Принято»
rejected	сервер	отклонено	«Ошибка»

- Переходы: draft → complete → queued → uploaded → accepted / rejected.
- Из rejected возможен возврат в draft для исправления.
- Что это закрывает:

п. 4.1 ТЗ «событие не сохраняется до окончания съёмки» → выполняется на уровне complete. Черновик виден водителю как «Не закончено», логисту не уходит.

- Краш или севшая батарея → черновик восстанавливается с диска.

7.2. Идемпотентность

- `client_event_id` (UUIDv7) на клиенте — уникальный индекс (`client_event_id`, `device_id`).
- Idempotency-Key на API.
- `attachment_id` на файлах.
- Повторная отправка ничего не дублирует — критично для плохой связи.

7.3. Приоритеты загрузки

- Сначала превью и метаданные — логист видит событие через минуты.
- Потом оригиналы — уже в фоне.

7.4. Очистка на клиенте

Локальные файлы удаляются только после `state='accepted'` на сервере. Квота под фото (например, 2 ГБ) + предупреждение водителю. При нехватке места — сначала отправляем, потом чистим.

7.5. Разделение состояний

Состояние	Значения
Событие	<code>draft / complete / queued / uploaded / accepted / rejected</code>
Доставка	<code>pending / sending / sent / error / retrying</code>
Рейс	<code>assigned / in_progress / pending_logistician / closed / cancelled</code>

- Это три разных набора статусов. Смешивать их — классический источник багов.

7.6. Фоновые задачи и очередь

- При приёме события сервер в одной транзакции сохраняет:

запись в `events`,

- задачу в `outbox`.

Воркер читает `outbox`, отправляет, пишет `notification_log`, ретраит с `backoff`. Этот шаблон называется `transactional outbox`. В начале очередь можно хранить в PostgreSQL; позже подключить брокер сообщений.

8. Время, координаты, доказательность

8.1. Время как набор атрибутов

T3 содержит противоречие: «время по серверу» ↔ офлайн-режим. В офлайне сервера нет, значит доверять можно только локальным часам, которые водитель может перевести.

- Решение — время как набор атрибутов, а не одно поле.

Атрибут	Смысл
<code>device_time_utc</code>	время устройства на момент события
<code>device_tz_offset_min</code>	смещение зоны устройства (для отображения)
<code>elapsed_realtime_ms</code>	монотонные часы (<code>SystemClock.elapsedRealtime</code>) — защита от перевода времени

Атрибут	Смысл
server_time_utc	время сервера в момент приёма события
received_at_utc	когда сервер принял (всегда достоверно)
clock_skew_ms	расхождение, вычисляется при синхронизации
time_trust	high / skewed / unknown — по нему логист видит достоверность

UI показывает местное время устройства; веб и Telegram — время события в зоне точки, с флагом расхождения. Это закрывает п. 5.1 ТЗ и одновременно даёт доказательность.

- Порог clock_skew_ms: 5 минут.

8.2. Геолокация через порт

- ТЗ содержит противоречие: FusedLocationProvider (GMS) ↔ Huawei без Google Play Services.
- Решение — интерфейс LocationProvider + реализации, выбираемые на старте приложения:

Реализация	Платформа	Устройства
GmsLocationProvider (FusedLocationProviderClient)	Android	Samsung, Xiaomi, Realme, Honor с GMS
HmsLocationProvider (Huawei Location Kit)	Android	Huawei с HMS
PlatformLocationProvider (Android LocationManager)	Android	универсальный fallback, работает всегда
IosLocationProvider (Core Location)	iOS	все iPhone (позже)

- Методы порта:
- `getOneShotLocation(timeoutSeconds: Int): Location?` — одна точка (при событии).
- `startPeriodicTracking(intervalMinutes: Int): Flow<Location>` — периодический трекинг (ГЛОНАСС-аналог).
- `stopPeriodicTracking()` — остановка.
- `getAccuracy(): Int` — точность последней точки.
- Политика получения точки: агрессивный one-shot с таймаутом 20–30 с, учёт ассурасы, деградация при неудаче.
- То же самое — для push-уведомлений (FCM ≠ HMS ≠ APNs) и сканера документов (ML Kit). Всё за портами.

8.3. Периодический трекинг (ГЛОНАСС-аналог)

Параметр	Android	iOS
Планировщик	WorkManager + foreground service	BGTaskScheduler
Интервал	15 минут (минимум)	15 минут
Когда работает	Только во время рейса	Только во время рейса
Разрешение	ACCESS_BACKGROUND_LOCATION	NSLocationAlwaysAndWhenInUseUsageDescription
Согласие	Отдельное (consent_type='tracking')	Отдельное

- Стоимость: 0 Р. Бесплатные методы.
- Преимущества:

- Видно маршрут движения.
- Можно связать событие с точкой трека.
- Доказательная база усиливается.

8.4. Водяной знак

- Дата, время, координаты, номер рейса — на каждое фото.

Важно: водяной знак не доказывает подлинность координат и времени. Это отдельная проблема, решается подписью и ЭТрН на этапе 16.

9. Уведомления и интеграции

9.1. Каналы уведомлений

Канал	Платформа	Статус
Telegram	сервер	Основной (логист + dev-bot)
FCM	Android	Push
HMS	Android (Huawei)	Push
APNs	iOS	Push (позже)
SMS	сервер	Опционально
E-mail	сервер	Опционально
Web-push	веб	Опционально

- Правила:
- Уведомления идут через очередь с ретраями и журналом доставки.
- Веб-кабинет обязан показывать те же события независимо от Telegram.

12 ракурсов не влезают в один Telegram-альбом — несколько сообщений или ссылка в кабинет.

9.2. Telegram как канал доставки, не хранилище

- Проблема: Telegram — внешняя зависимость. Лимиты Bot API: около 30 сообщений/с, альбом до 10 фото, файл до 50 МБ. Риск блокировок.
- Решение:
- Уведомления идут через очередь с ретраями и журналом доставки.
- Веб-кабинет обязан показывать те же события независимо от Telegram.

12 ракурсов — несколько сообщений или ссылка в кабинет.

- Иначе «логист видит в реальном времени» зависит от стороннего сервиса.

9.3. Transactional outbox

При приёме события сервер в одной транзакции сохраняет запись в events и задачу в outbox. Воркер читает outbox, отправляет, пишет notification_log, ретраит с backoff.

9.4. Юридический момент

- Фото в Telegram = копия вне основного хранилища. Это надо явно прописать в ТЗ и согласовать с заказчиком по 152-ФЗ.
- Решение заказчика: сами фото. Риск принят заказчиком (см. ADR-0010).

9.5. RuStore

- RuStore — канал обновления для Android (по решению заказчика).
- Публикация APK в RuStore — после пилота.
- В app_releases поле channel: direct / rustore.
- Порт UpdateChannel содержит RuStoreChannel (реализация позже).

9.6. Точки расширения

/integrations/etn, /integrations/1c, /integrations/sms — заложены сразу. Добавление нового канала (SMS, e-mail, web-push) = новый адаптер + шаблон в notification_templates.

10. Структура репозитория

10.1. Общая структура (монорепо)

```

/reiscontrol
  /apps
    /android      # Kotlin, Compose, multi-module
    /ios          # Swift, SwiftUI (позже)
    /web          # React + TS + Vite (SPA)
    /telegram     # боты-нотификаторы
  /backend
    /apps
      /api        # FastAPI: роутеры v1/v2
      /worker     # обработчики очереди
      /scheduler  # cron: реконсиляция, lifecycle, проекции
    /domain       # сущности, правила (без БД, без FastAPI)
      /trips
      /events
      /checklists
      /attachments
      /access
      /rules
    /application  # use-cases + порты
    /infrastructure # адаптеры: postgres, s3, ymq, telegram
    /integrations # точки расширения: /etn, /1c, /sms
    /contracts    # openapi.yaml, JSON Schema
    /migrations   # Alembic (expand/contract)
  /database
    /migrations
  /infra          # Terraform: bucket, PG, YMQ, secrets
  /tests
    /unit
    /integration  # testcontainers
    /contract     # OpenAPI
    /e2e         # «рейс от загрузки до закрытия»
  /docs
    /01-requirements # ТЗ
    /02-architecture # Архитектура
    /03-adr          # ADR-0001..0011
    /04-db           # Схема БД
    /05-api          # openapi.yaml
    /06-schemas     # JSON Schema
    /07-wireframes   # Макеты
    /08-decisions    # Журнал итераций

```

```
/config
/checklists          # «золотые» конфигурации
```

10.2. Android — модули

```
:app                навигация, DI-граф, сборка сценариев
:core:design          тема, крупные элементы, high-contrast
:core:network        Retrofit/OkHttp, интерсепторы, auth
:core:database        Room, DAO, миграции, шифрование
:core:sync            Outbox, WorkManager, идемпотентность
:core:location        LocationProvider + {Gms, Hms, Platform}
:core:camera          CameraX, съёмка «только сейчас», EXIF
:core:media           водяной знак, превью, хеширование
:core:config          server-driven конфиг, интерпретатор шагов
:core:rules           локальная предпроверка правил завершения
:core:logging         локальный журнал, dev-bot, crash
:core:update          проверка версии, APK, подпись
:core:security        токены, PIN/биометрия
:core:common          Result, время, UUIDv7, корутины
:feature:auth
:feature:orders
:feature:trip
:feature:event
:feature:checklist
:feature:closing
:feature:diagnostics
:feature:settings
```

Правило зависимостей: feature-модули не зависят друг от друга, только от core:*. Связи — через навигационные контракты в :app.

10.3. iOS — модули (закладывается)

```
:app                навигация, DI
:core:design          тема, крупные элементы
:core:network        URLSession, интерсепторы, auth
:core:database        Core Data / Realm
:core:sync            Outbox, BGTaskScheduler
:core:location        Core Location
:core:camera          AVFoundation
:core:media           водяной знак, превью
:core:config          server-driven конфиг
:core:rules           локальная предпроверка
:core:logging         локальный журнал
:core:update          TestFlight / App Store
:core:security        Keychain
:core:common          Result, время, UUIDv7
:feature:auth, orders, trip, event, checklist, closing, diagnostics, settings
```

10.4. Backend — правило зависимостей

apps → application → domain

infrastructure → application/domain

domain не знает ни про SQL, ни про HTTP. Пакеты внутри слоя — по бизнес-возможностям (trips, events, checklists), а не по типам файлов.

11. Версии и совместимость

11.1. Независимые версии

Что	Где хранится
App version	X-App-Version заголовок

Что	Где хранится
Platform	X-Platform (android / ios)
API version	URL /api/v1, /api/v2
Config version	config_snapshots на рейс
Event schema version	payload_schema_version
DB schema version	Alembic

11.2. Правила

- Аддитивность контрактов. Изменения только добавлением полей. Удаление = депрекация на 2 релиза → отключение флагом → удаление через полгода.

Минимальная версия. Таблица app_releases (latest, min_supported, rollout%, platform, channel).

Ответ 426 Upgrade Required — только в исключительных случаях, с обоснованием в ADR.

- Старые версии работают. Проекция конфига под версию клиента. Даже устаревшая версия должна уметь выгрузить накопленные материалы (многодневные рейсы).
- Контракт-first. Единый openapi.yaml — источник правды. Из него генерируются клиентские модели для Android и TS-типы для веба + contract-тесты в CI.
- Один API для Android и iOS.

11.3. Каналы обновления

Платформа	Канал	Метод	Статус
Android	Direct APK	Object Storage + SHA-256 + подпись	Основной
Android	RuStore	Публикация в RuStore	После пилота
iOS	TestFlight	Для пилота	Позже
iOS	App Store / Enterprise	Production	Позже

- Правила:
- Обновление не принудительное, кроме случая min_version.
- Ответ 426 Upgrade Required — только в исключительных случаях, с обоснованием в ADR.
- Для Android: GET /app/releases/latest?platform=android&channel=direct.
- Для iOS: GET /app/releases/latest?platform=ios&channel=testflight.

11.4. Миграции

- Только expand/contract:
- Добавили поле.
- Пишем в оба.
- Бэкфил.
- Переключили чтение.
- Удалили старое.
- Никогда «big bang» миграция со стопом.

12. Этапы работ

Этап 0 (3–5 дней): документы и каркас

- ADR-0001..0011.

- Схема БД.
- `openapi.yaml`.
- JSON Schema примитивов.
- «Уточнения к ТЗ» (или включены в ТЗ v4.0).
- Репозиторий GitHub.
- Yandex Cloud: аккаунт, Object Storage (dev / staging / prod), PostgreSQL (dev / staging / prod), YMQ, Lockbox.
- Telegram-боты.

Домен + SSL.

- Каркас репозитория.
- CI/CD.
- Шаблоны (README, AGENTS, CONVENTIONS, ROADMAP, ARCHITECTURE, `.cursorrules`).

Этап 1 (7–10 дней): walking skeleton

- Цель — сквозной сценарий от заявки до уведомления.
- Авторизация → заявка → «Я на месте» → GPS → камера → водяной знак.
- Офлайн-очередь сразу (не этапом 4).
- Синхронизация → Telegram → веб-кабинет (одна страница).
- CI/CD с первого дня.
- Что заложить сразу, даже если не используется:
- `cargo_units` и `trip_participants` в схеме.
- `config_snapshots` и `event_types` вместо хардкода «4 кнопок».
- `client_event_id` + `outbox` на клиенте.
- `attachments` с `sha256`, `kind`, `version_of`, двухфазной загрузкой.
- `events` как append-only с `payload_jsonb` + `schema_version`.
- Заголовки версии + `app_releases` с `min_supported`, `platform`, `channel`.
- Feature flags + очередь уведомлений (один адаптер Telegram, но через порт).
- `location_tracks` (новая таблица).
- `user_consent`s (согласия на ПДн, гео, трекинг).
- Поля `platform` везде, где применимо.

Этапы 2–3 (10–15 дней): чек-лист и документы — как данные

- Конфигуратор в веб-кабинете.
- JSON Schema для примитивов.

12 ракурсов, обязательные документы.

Этапы 4–5 (8–12 дней): надёжность

- Полевой тест офлайна: перезапуск, обрывы, частичные загрузки.
- Завершение рейса: правила + трек-номер + подтверждение логиста.
- Периодический трекинг: полевой тест.

Этапы 6–7 (8–12 дней): кабинет, уведомления, логи

- Полноценный веб-кабинет.
- Просмотр треков (карта).
- Доработка ботов.
- Страница логов.
- Метрики и алерты.

Общий срок: 2–2,5 месяца — как пилот. Не как production. Надёжность офлайна подтверждается полевым тестом (этап 7).

13. Критерии приёмки

13.1. Из ТЗ (базовые)

- Водитель проходит цепочку «Я на месте» → GPS → камера → фото → отправка.
- Логист получает событие с координатами и фото в Telegram.
- Разработчик получает логи в Telegram.
- Фото хранится в папке рейса.
- Водяной знак виден на фото.

12 ракурсов на каждую машину, событие не сохраняется до окончания.

- Обязательные документы, рейс нельзя завершить без фото и трек-номера.
- Офлайн: событие сохраняется локально, отправляется автоматически.

13.2. Дополнительные (критичные)

- Обновить приложение с незагруженными фото → проверить сохранность.
- Повторить отправку операции → получить одно событие (идемпотентность).
- Изменить чек-лист во время офлайн-рейса → сохранить прежние требования рейса.
- Восстановить БД и файлы из резервной копии → открыть полный рейс.
- Периодический трекинг — точки записываются каждые 15 минут во время рейса.
- Согласие на ПДн — при первом входе, отдельно на геолокацию и трекинг.

13.3. Из требований гибкости

- Изменение чек-листа, ракурсов или списка документов не требует релиза приложения.
- Добавление нового типа события не требует переписывания системы.
- Изменение структуры хранения не приводит к потере сохранённых данных.
- Старая версия приложения продолжает работать после обновления сервера.
- iOS-клиент добавляется без переделки API и схемы БД.

14. Открытые вопросы к заказчику

- Уже отвечено:

Вопрос	Ответ	Источник
Экипаж, подмены	trip_participants	ADR-0009, схема БД
GPS не поймал	Деградация	ADR-0006
Досъёмка из галереи	Запрещена	ТЗ п. 4.1
Устройства	Личные	ADR-0010
Логисты	2–3, скопы	ADR-0009
Выгрузки	ZIP по рейсу	ТЗ
Бюджет	~5–7 тыс Р/мес	ТЗ п. 10
152-ФЗ	Заказчик	ADR-0010
Срок хранения фото	1 год	ADR-0010
Фото в Telegram	Сами фото, риск принят	ADR-0010
Согласие на геолокацию	Отдельное	ADR-0010

- Остаются открытыми:

Сколько машин в среднем и максимум в одном рейсе?

Сколько рейсов в месяц через год?

- Юридическая сила доказательной базы (этап 16).
- Тарифы мобильного интернета водителей.

14.1. Расчёт объёмов (важно)

12 ракурсов × 8 машин + ~10 документов ≈ 110–130 фото на рейс.

- При сжатии до 1920 px / q80 → ~0,5 МБ на фото + превью ~30 КБ → 65–80 МБ на рейс.

30 рейсов/мес → ~2 ГБ/мес.

- Бесплатный 1 ГБ Object Storage кончится на 12–15 рейсах.

200 рейсов/мес → ~13 ГБ/мес, ~160 ГБ/год + исходящий трафик на просмотр логистом.

Вывод: бесплатный тариф — только демо. Нужны серверное сжатие, генерация превью, лимит разрешения на клиенте, lifecycle-политика (холодное хранилище через 90 дней) и расчёт бюджета до старта. Иначе «пилот бесплатно» ломается на втором месяце.

15. Антипаттерны

Антипаттерн	Почему
Чистый event sourcing	Сложно, команда не вытянет за 2 месяца. Гибрид даёт 80% пользы за 20% усилий.
YDB на пилоте	Дороже по времени разработки, меньше экспертизы. PostgreSQL + JSONB закрывает всё.
Универсальный конструктор чек-листов	Не нужен на старте. 7 примитивов покрывают 90% сценариев.
Офлайн «этапом 4»	От него зависят ID, статусы, синхронизация. Только с 1-го этапа.
Telegram как единственный канал	Внешняя зависимость. Веб-кабинет обязателен.
Cloud Functions как «весь бэкэнд»	PDF, ресайз, экспорт ZIP — только воркеры.
Хардкод «4 кнопок событий»	Этап 2 сломает. Только через event_types.
Принудительное обновление	Только для min_version, с обоснованием в ADR.
UPDATE поверх accepted-событий	Только новые записи + corrects_event_id.
Big bang миграции	Только expand/contract.
Привязка API к платформе	Android и iOS используют один API.
Хардкод канала обновления	direct / rustore / testflight / appstore — в app_releases.
Постоянный GPS-трекинг	ТЗ п. 12 исключает. Только периодический и при событии.
Трекинг вне рейса	ADR-0010: только во время рейса.

16. Итоговая трассировка на требования ТЗ

Требование ТЗ	Механизм в гибриде	Источник
6.2 Обновления без одновременного обновления	аддитивный API + min_supported + свой канал APK	гибкость
6.2 Поэтапное включение/отключение	feature flags + kill switch	гибкость
6.2 Новые типы событий/документов/ракурсов	справочники + примитивы + JSON Schema	гибкость
6.2 Смена состава и порядка чек-листа	config-as-data + конфигуратор	гибкость
6.2 Смена структуры хранения без потери	append-only + проекции + expand/contract	гибкость
6.2 Новые каналы и интеграции	порты + адаптеры + очередь + шаблоны	гибкость
6.3 Не ломать сохранённые рейсы	config_snapshot на рейс + иммутабельные события	гибкость
6.4 Сопровождаемость	слоистая структура, ADR, contract-first, тесты, IaC	гибкость
6.5 Конфигурируемость	конфигуратор + правила завершения как DSL	оба
7.2 Разграничение доступа	RBAC + trip_participants + access_scopes	гибкость
7.2 Выгрузки	export_jobs + воркер + виртуальная раскладка	гибкость
7.3 Восстановление и целостность	PITR + версионирование бакета + sha256 + реконсильация	оба
Прагматизм пилота	7 примитивов, PostgreSQL, разумные сроки	прагматизм
Дополнительные критерии приёмки	обновление с неотправленными фото, повтор, офлайн-чек-лист, бэкап	прагматизм
Юридический блок	152-ФЗ, копия фото в Telegram	прагматизм
Мультиплатформенность	X-Platform, platform в БД, порты	новое
RuStore как канал	channel в app_releases	новое
Периодический трекинг (ГЛОНАСС)	location_tracks, startPeriodicTracking	новое

Что изменилось в v1.1

№	Изменение	Раздел
1	Добавлена мультиплатформенность (Android + iOS)	4.1, 5.6, 8.2, 10.3, 11
2	iOS-стек описан	4.1, 5.6, 10.3
3	RuStore как канал обновления	9.5, 11.3

№	Изменение	Раздел
4	Периодический трекинг (ГЛОНАСС-аналог)	5.8, 8.3
5	location_tracks в схеме БД	5.4, 5.8
6	user_consent в схеме БД	5.4
7	Поля platform везде, где применимо	5.3, 11.1
8	Каналы обновления: direct / rustore / testflight / appstore / enterprise	11.3
9	APNs как push-канал	4.1, 9.1
10	Новые антипаттерны	15
11	Обновлённая трассировка	16

- Конец документа.